

Digital Image Synthesis Project #2:

Uniform Grid

R14942096 林祐群

Uniform Grid & Two-level Uniform Grid for PBRTv4

Goal: Implement (two-level) uniform grids as an acceleration structure in pbrt v4. Uniform grids were supported in pbrt v2 but were removed after pbrt v3 due to their inferior performance. The primary goal of this project is to help become familiar with the architecture of pbrt v4.

1. Implementation

High level idea for uniform grid and two-level uniform grid implementation:

i. Grid Construction:

For the uniform grid, the construction begins by computing the bounding box of all primitives. The entire scene space is then discretized into a 3D grid.

Each primitive is inserted into all voxels overlapped by its bounding box. The mapping from 3D voxel coordinates to linear memory is done through flattening:

$$index = x + n_x(y + n_y z)$$

For the two-level uniform grid, the construction is extended as follows:

A top-level grid is first constructed in the same way as the single-level grid. For each cell: if the number of primitives exceeds a threshold (`minPrimsForSubgrid`), a subgrid is constructed locally. Otherwise, the primitive list is kept as-is. This design allows high-density regions to be further subdivided while avoiding unnecessary overhead in sparse regions.

ii. Ray Traversal:

Ray traversal for both structures is based on **3D DDA** (Digital Differential Analyzer):

The ray first intersects the global bounding box. It then steps through voxels incrementally along the ray direction.

At each voxel:

- Uniform Grid: directly tests intersection with all primitives in the cell.

- Two-Level Grid: If the cell has a subgrid → recurse into subgrid traversal. Else → test primitives directly.

Early termination is applied whenever a closer hit is found (updating tMax).

iii. Determine accelerator:

The accelerator structure is selected via scene configuration and mapped using `CreateAccelerator()`. Supported accelerators include:

[bvh, kdtree, uniformgrid, twoleveluniformgrid]

This ensures all four methods are evaluated under identical rendering conditions.

Modified files of source code:

File	Description of Changes
primitive.h	Added TwoLevelUniformGridAggregate and UniformGridAggregate into TaggedPointer type system
aggregates.cpp	Implemented construction and traversal logic
aggregates.h	Declared class structure and internal cell representation
pbrt_lib.vcxproj	Added new source files into build system
CMakeLists.txt	Registered new accelerator for compilation

Concise overview of implementation:

i. primitive.h:

Extended the Primitive variant type to include the newly implemented TwoLevelUniformGridAggregate and UniformGridAggregate, ensuring they can be used interchangeably with other accelerators.

ii. aggregates.cpp:

Implemented:

- Grid construction (top-level and subgrid for two-level uniform grid)
- DDA traversal logic
- Intersection routines (Intersect, IntersectP)
- Accelerator registration

iii. aggregates.h:

Defined:

- Class interface
- Internal CellData structure containing primitive indices and optional subgrid pointer

iv. pbrt_lib.vcxproj:

Updated Visual Studio project file to include new source files and ensure correct linking

v. CMakeLists.txt:

Registered new files and ensured the (two-level) grid is compiled as part of PBRT

2. Experimental Setup

Operating system: Windows 11 Pro

CPU information: Intel Core i7-13700 (2.10 GHz)

Sample per pixel: 128, 512, ...; depends on scenes

Memory: 32 GB RAM

3. Own-created scenes

i. Shear shape from project #1 [.\scenes\broken-normals.pbrt]:

This scene reuses geometry from Project #1 with non-uniform transformations (scaling and rotation), introducing moderate geometric complexity and non-axis-aligned shapes. It serves as a baseline to verify correctness and consistency across different accelerators.

ii. Multiple balls in compacted space (high density) [.\scenes\accel-compare-dense]:

This scene consists of many spheres tightly packed within a small region, creating high primitive density and heavy overlap. The purpose is to (a) stress-test traversal efficiency when many primitives exist within the same voxel, and (b) evaluate effectiveness of two-level subdivision in reducing brute-force intersection.

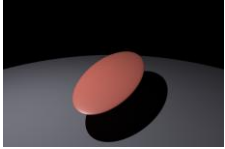
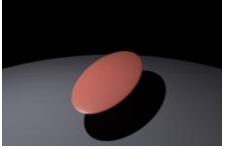
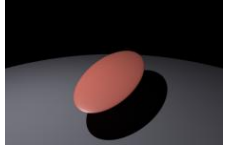
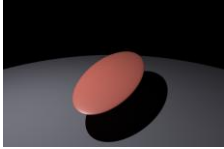
iii. A teapot in a stadium [.\scenes\accel-compare-stadium]:

This scene simulates the classic “teapot in a stadium” scenario, where extremely large bounding volume and very small region with dense geometry (teapot) is presented. The purpose is to highlight weaknesses of uniform grids in sparse scenes and evaluate how much two-level grid mitigates empty space traversal overhead.

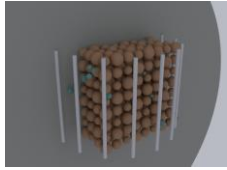
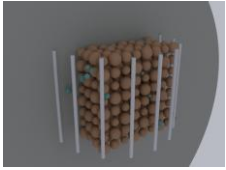
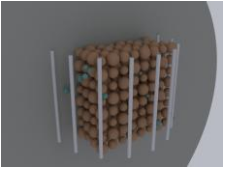
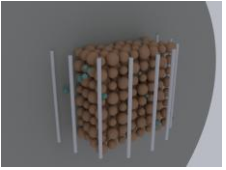
4. Quantitative & Qualitative Results

I’ve tested my implementation of (two-level) uniform grid in for five scenes, three of my own-created scenes and two scenes from public pbrtv4 GitHub repository.

1. My scene 1 (from project #1):

	BVH	Kdtree	Uniform Grid	Two-level Uniform Grid
Result				
Construct (ms)	14	9	13	14
Render (ms)	39686	62747	85146	45596
Wall (ms)	39701	62756	85159	45601

2. My scene 2 (dense):

	BVH	Kdtree	Uniform Grid	Two-level Uniform Grid
Result				
Construct (ms)	15	15	12	13
Render (ms)	6330	7911	15935	13896
Wall (ms)	6345	7927	15947	13910

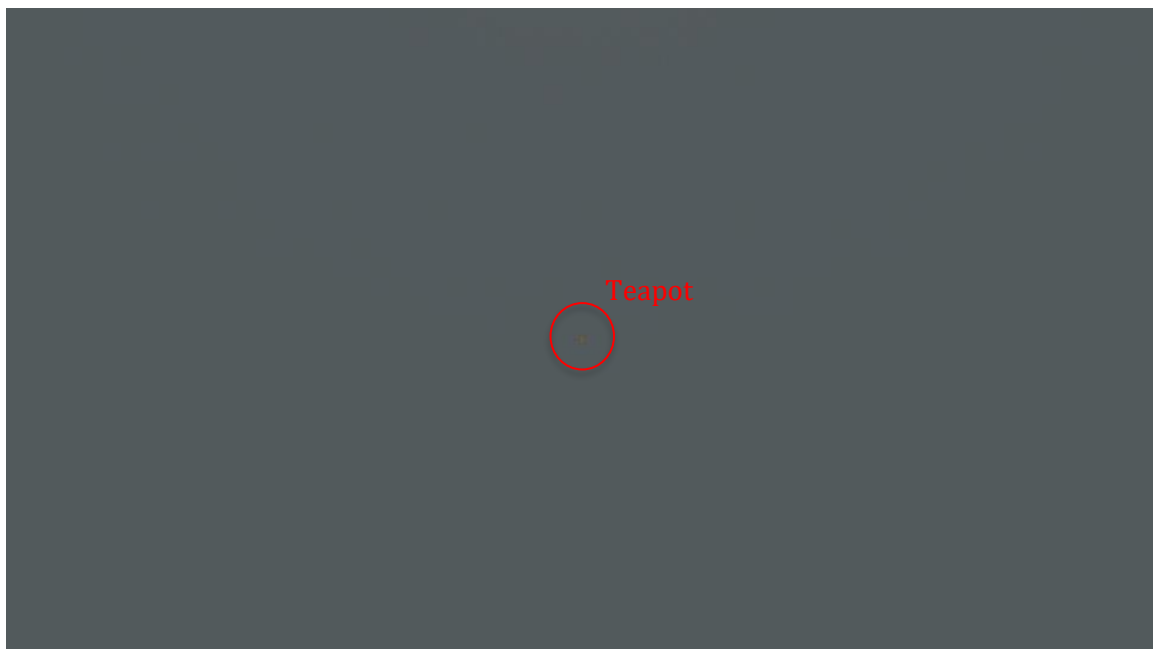
3. My scene 3 (teapot in stadium):

Teapot ply file:

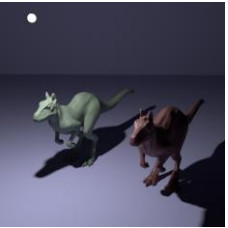


	BVH	Kdtree	Uniform Grid	Two-level Uniform Grid
Result				
Construct (ms)	16	16	15	17
Render (ms)	6873	13575	225818	186422
Wall (ms)	6889	13592	225834	186439

Enlarge image:



4. Killeroo-simple:

	BVH	Kdtree	Uniform Grid	Two-level Uniform Grid
Result				
Construct (ms)	29	18	20	23
Render (ms)	39507	94578	894570	227579
Wall (ms)	39536	94597	894590	227602

5. Bistro-boulangerie:

	BVH	Kdtree	Uniform Grid	Two-level Uniform Grid
Result				
Construct (ms)	1307	253	257	256
Render (ms)	750677	1591462	12903799	5045642
Wall (ms)	751983	1591715	12904056	5045889

5. Analysis

From the experimental results, several key observations can be made:

1. BVH consistently achieves the best performance

Across all scenes, BVH provides the lowest rendering time due to its hierarchical bounding volume structure, which efficiently prunes ray intersections. This also explains the reason why pbrtv4 choose BVH as its default accelerator.

2. KD-tree performs well but is less stable

KD-tree generally performs better than uniform grids but worse than BVH, especially in scenes with irregular geometry distribution. For instance, in scene 1, the wall time for KD-tree is even slower than two-level grid.

3. Uniform grid suffers in non-uniform scenes

In dense scenes, performance degrades due to excessive primitive checks per voxel

In sparse scenes (e.g., teapot in stadium), traversal cost becomes "extremely high" due to empty space

4. Two-level uniform grid improves performance

Two-level grid method successfully reduces primitive count per voxel in dense regions, and achieves noticeable speedup over single-level grid (up to ~4x in some cases). However, it still slower than BVH/KD-tree in most scenarios.

5. Trade-off insight

Uniform grid → simple but inefficient

Two-level grid → improved locality handling

BVH/KD-tree → better global adaptability

6. Conclusion

In this project, both uniform grid and two-level uniform grid acceleration structures were implemented in PBRT v4 and evaluated against BVH and KD-tree.

The two-level uniform grid successfully improves upon the limitations of the single-level grid by introducing adaptive local refinement. It significantly reduces intersection cost in high-density regions. However, it still cannot outperform hierarchical structures like BVH in most real-world scenarios.

The results demonstrate that scene distribution plays a critical role in choosing the appropriate acceleration structure.

7. Appendix

Since I also compiled the GPU version for PBRTv4, I validated the dramatic improvement of acceleration brought by NVIDIA OptiX SDK. It's often almost intractable for CPU version to render some scenes (such as Dam Break, Disney Cloud, Landscape, and so on) even with BVH or KD-tree, not to mention legacy uniform grid method.

References

1. Teapot mesh: <https://github.com/gnomeby/canvas3D/blob/master/teapot.ply>
2. PBRTv2: <https://github.com/mmp/pbrt-v2/tree/master/src/accelerators>
3. PBRTv4 scenes: <https://github.com/mmp/pbrt-v4-scenes?tab=readme-ov-file#barcelona-pavilion>